- build_association(attributes = {})
- create_association(attributes = {})
- create_association!(attributes = {})

In all of these methods, association is replaced with the symbol passed as the first argument to belongs_to. For example, given the declaration:

```
class Order < ActiveRecord::Base
  belongs_to :customer
end
```

Each instance of the Order model will have these methods:

```
customer
customer=
build_customer
create_customer
create_customer!
```

When initializing a new has_one or belongs_to association you must use the build_prefix to build the association, rather than the association.build method that would be used for has_many or has_and_belongs_to_many associations. To create one, use the create_ prefix.

4.1.1.1 association(force_reload = false)

The association method returns the associated object, if any. If no associated object is found, it returns nil.

```
@customer = @order.customer
```

If the associated object has already been retrieved from the database for this object, the cached version will be returned. To override this behavior (and force a database read), pass true as the force_reload argument.

4.1.1.2 association=(associate)

The association= method assigns an associated object to this object. Behind the scenes, this means extracting the primary key from the associate object and setting this object's foreign key to the same value.

```
@order.customer = @customer
```

4.1.1.3 build_association(attributes = {})

The build_association method returns a new object of the associated type. This object will be instantiated from the passed attributes, and the link through this object's foreign key will be set, but the associated object will *not* yet be saved.

```
@customer = @order.build_customer(customer_number: 123,
                                  customer_name: "John Doe")
```

4.1.1.4 create_association(attributes = {})

The create_association method returns a new object of the associated type. This object will be instantiated from the passed attributes, the link through this object's foreign key will be set, and, once it passes all of the validations specified on the associated model, the associated object *will* be saved.

```
@customer = @order.create_customer(customer_number: 123,
                                   customer_name: "John Doe")
```

4.1.1.5 create_association!(attributes = {})

Does the same as create_association above, but raises ActiveRecord::RecordInvalid if the record is invalid.

4.1.2 Options for belongs_to

While Rails uses intelligent defaults that will work well in most situations, there may be times when you want to customize the behavior of the belongs_to association reference. Such customizations can easily be accomplished by passing options and scope blocks when you create the association. For example, this association uses two such options:

```
class Order < ActiveRecord::Base
  belongs_to :customer, dependent: :destroy,
    counter_cache: true
end
```